C-cured User's Manual

A cure for the in-C-cureness of the C language

Contents

1 Overview **2**

2 Lexical Elements **4**
- 2.1 Identifiers . . . 4
- 2.2 Literals . . . 4
- 2.3 Semicolons . . . 4

3 Types **5**
- 3.1 Primitive types . . . 5
- 3.2 Array types . . . 5
- 3.3 Record types . . . 5
- 3.4 Type casting . . . 6

4 Expressions and Operators **7**

5 Statements **8**
- 5.1 Variable and constant declaration . . . 8
  - 5.1.1 Scope . . . 8
- 5.2 Assignment . . . 8
- 5.3 Input and output . . . 9
- 5.4 Control flow statements . . . 9
- 5.5 Return . . . 10

6 Functions **11**

7 Null-Reference Checking **12**

8 Quick Reference **13**
- 8.1 Complete Example . . . 14

Chapter 1

Overview

C-cured is a statically-typed, imperative language, and interpreted with strong safety guarantees. Though the language is interpreted, programs are first analyzed for errors (which are reported to the user) before being properly executed. In this manual, this analysis stage is referred to as "pre-execution". Whenever possible, the language catches possible errors during pre-execution, not during execution proper. We distinguish between these two stages because errors are ideally caught during pre-execution, before the program is in a situation where correctness is critical.

Syntactically, it shares many similarities with the C language. Its main differences from C are as follows.

- Immutable constants are declared with `let`
- Booleans are separate from integers and conditions must always be boolean
- There are no pointers; records and arrays are passed by reference automatically, while primitives are always passed by value
- The types of **all** expressions are known during pre-execution
- Type inference is supported for variable and constant declarations
- It is enforced during pre-execution that variables are not used before they are given values (except arrays, see section 7)
- Non-void functions must always return a value

A program consists of three sections in this fixed order.

```
record Point  { x: float, y: float }
record Circle { center: Point, radius: float }

float dist(p: Point) { ... }
bool  intersects(a: Circle, b: Circle) { ... }

main: {
    var p: Point;
    p.x = 0.0;
    p.y = 0.0;
    print(dist(p));
```

```
}
```

Record type declarations come first, followed by function declarations, then the `main` block. The first two sections may be empty, but the `main` block must always be present (though its body may be empty).

Chapter 2

Lexical Elements

2.1 Identifiers

An identifier is a name given to a variable, constant, function, or record type. It must begin with a letter (`a`–`z`, `A`–`Z`) or an underscore (`_`), followed by any number of letters, digits (`0`–`9`), and underscores. Identifiers are case-sensitive. An identifier may not be the same as a reserved word, below is the full list of reserved words.

```
let    var    record  main   if      else
for    while  repeat  until  return  void
print  scan   cast    arr    true    false
int    float  bool    char   str
```

2.2 Literals

The language supports literals for all the primitive types (`int`, `float`, `bool`, `char`, and `str`). An `int` literal is a sequence of decimal digits, optionally preceded by a minus sign: `0`, `42`, `-7`. A `float` literal is the same but includes a decimal point with at least one digit following the decimal point: `3.14`, `-0.5`. The boolean literals are `true` and `false`. A `char` literal is a single printable ASCII character enclosed in single quotes: `'a'`, `'Z'`. The newline character is also valid as a `char` literal and is written `'\n'`. A `str` literal is any sequence of printable ASCII characters enclosed in double quotes: `"hello"`.

2.3 Semicolons

Most statements are terminated by a semicolon. This applies to variable and constant declarations, assignments, `print`, `scan`, `return`, and bare function calls. It also applies to the `repeat` loop as a whole, which is terminated by `until (condition);`. Unlike in C, block constructs that end with a closing brace, e.g. `if`, `while`, `for`, type declarations, function declarations, do *not* have a semicolon after the closing brace (except repeat loops).

Chapter 3

Types

3.1 Primitive types

The five primitive types are `int`, `float`, `bool`, `char`, and `str`. They are always passed by value.

3.2 Array types

An array type is written as follows. The base type is written first, followed by `arr` and a single pair of brackets containing the dimension sizes as a non-empty, comma-separated list. Any primitive or record type may serve as the base type.

```
int arr[10]
int arr[3, 4]
Point arr[5]
```

Dimension sizes must be integer expressions. They are fixed at the time of declaration and cannot change afterwards.

Element access uses the same bracket syntax. For an n-dimensional array, exactly n comma-separated integer indices are required.

```
m[1, 2] = 7;
var e = m[0, 0];
```

A `str` may also be indexed with a single `int` to retrieve the `char` at that position. Array bounds are checked during execution. Arrays are passed by reference.

3.3 Record types

Records are declared at the top level and passed by reference. All field types must already have been declared before the record that uses them. A field may not have the same type as the enclosing record (no recursive types), and a field name must differ from the record's own name. Fields may be of any type, including other record types, and array types.

```
record Vec2    { x: float, y: float }
record Segment { a: Vec2, b: Vec2 }
record Matrix  { data: float arr[4, 4], rows: int, cols: int }
```

The `Matrix` example shows an array-typed field. Array fields (and arrays in general) have special behavior with respect to null-reference checking, as described in Section 7.

Fields are accessed with the usual dot syntax. Field accesses can be chained to access nested attributes.

```
var s: Segment;
s.a.x = 0.0;
```

3.4 Type casting

The built-in function `cast` converts between primitive types. The target type is passed as a string literal.

```
cast(expr, "targetType")
```

The only valid conversions are:

Source	Valid target types
int, float	int, float, bool, char, str
bool	int, float, bool, str
char	int, char, str
str	int, float, bool, str

There is one and only one instance where automatic type casting occurs, which is in arithmetic expressions between numbers (`int`s or `float`s) where operand types are not the same; the `int` is automatically promoted to `float`.

Chapter 4

Expressions and Operators

The following are considered expressions:

- Literals
- Variable/Constant names
- Array access (indexing an array)
- Field access (accessing an attribute of a record)
- (Non-void) Function invocation
- Binary and unary operations

Operators are shown below, ordered from highest to lowest precedence:

`Operator`	Meaning	Operand types
`!`	Logical NOT (unary)	`bool`
`* /`	Multiply, divide	`int` or `float`
`+ -`	Add/concatenate, subtract	`int`, `float`, or `str`+`str`
`< <= > >=`	Comparison (yields `bool`)	`int` or `float`
`== !=`	Equality (yields `bool`)	any identical types
`&&`	Logical AND	`bool`
`||`	Logical OR	`bool`

Parentheses may be used to control precedence in operations.

The arithmetic operators `*`, `/`, `-`, and `+` require both operands to be numeric (`int` or `float`). If one operand is `int` and the other is `float`, the `int` is implicitly promoted and the result is `float`; otherwise the result matches the operand type. The `+` operator additionally can accept two `str` operands, which performs string concatenation.

The relational operators always yield a `bool`. `<`, `<=`, `>`, `>=` require both operands to be numeric. Strings, booleans, and characters cannot be compared directly with these operators; convert to a numeric type with `cast` first if needed. The operators `==` and `!=` require both operands to be of the same type. For array types, the base-type, dimension, and dimension sizes must all be equal. Base-type or dimension mismatches are caught during pre-execution, while dimension size mismatches can only be caught during execution.

The logical operators `&&`, `||`, and `!` operate exclusively on `bool` values and yield `bool`.

Chapter 5

Statements

5.1 Variable and constant declaration

Constants and variables are declared while assigning a value, which can be any expression. Variables (but not constants) can also be declared without initialization, in which case the type of the variable must be specified and the variable must later be given a value before it can be referenced. The type need not (and should not) be specified when declaring with assignment, as C-cured will infer the type based on the value assigned.

```
var name: type;
var name = expression;
let name = expression;
```

Constants cannot be re-assigned and may not have a record or array type, since those types are always mutable.

Since there are no "record expressions" or "array expressions", record and array variables are always declared without assignment. The record variable's attributes and the array's elements are undefined until they are given values.

5.1.1 Scope

Scoping in C-cured follows C's block scoping rules. The global scope holds type and function names. Each function body and `main` introduce a local scope. `if/else` branches and loop bodies each introduce their own nested scope. Within a scope, all names declared within the scope or in ancestor scopes (the parent scope, parent's parent scope, and so on) can be accessed. Variables and constants cannot be re-declared where they are already in scope.

5.2 Assignment

Assignment statements have the following form.

```
lvalue = expression;
```

An lvalue is a variable, an array access, or a record field. The right-hand type must exactly match the left-hand type.

```
x = expr;
myArr[0] = expr;
myRecord.attr = expr;
```

Primitives are copied in assignment, whereas for records and arrays, the reference is copied. Note that when assigning an array to an array variable, the left and right-hand side arrays must have the same dimension and the same sizes in each dimension. The interpreter checks that dimensions are the same, but whether sizes are the same can only be known during execution, and so a size mismatch error will only be raised during execution.

5.3 Input and output

```
print(expression);
scan(lvalue);
```

The built-in functions `print` and `scan` allow console output and taking of user input, respectively. `print` evaluates its argument and writes the result to standard output. The argument may be of any non-void type. `scan` reads a line from standard input and stores it in the target variable, which must be a mutable `str` variable. To obtain numeric input, read into a `str` and convert with `cast`.

5.4 Control flow statements

```
if (condition) { ... }
if (condition) { ... } else { ... }

while (condition) { ... }

repeat { ... } until (condition);

for (i; rangeStart; rangeEnd; step) { ... }
```

All conditions must be of type `bool`. The `if` and `while` constructs behave as in C. `repeat/until` follows the `do/while` construct in C, evaluating its condition after the body.

The `for` loop in C-cured differs from C's `for`, instead being similar to Python's `for i in range()` construct. The first parameter is the identifier for the iterator variable, declared as an `int`, initialized to `rangeStart`, and scoped to the loop body, so it need not (and cannot) be declared separately. The next three parameters, `rangeStart`, `rangeEnd`, and `step` must all be expressions of type `int`; they are evaluated once before the loop begins. The iterator increases by `step` every iteration, and the loop continues iterating until the iterator exceeds `rangeEnd`. Note that `rangeEnd` is the **inclusive** end of the range.

5.5 Return

```
return;
return expr;
```

`return` is only valid inside a function body.

Chapter 6

Functions

```
returnType name(param1: type1, param2: type2) {
    return value;
}

void name(param1: type1) {
}
```

The return type (or `void`) precedes the function name with no separate keyword. Parameter names must be distinct and follow the rules for identifiers. All functions must be declared before `main`. A non-void function must have a reachable `return` that returns a value of the declared return type; a void function must not return a value.

Primitive types are passed by value. Records and arrays are passed by reference, so modifications inside the function are visible to the caller.

```
record Box { n: int }

void add(b: Box, x: int) {
    b.n = b.n + x;
}

main: {
    var myBox: Box;
    myBox.n = 10;
    add(myBox, 5);
    print(myBox.n);
}
```

After the call to `add`, `myBox.n` is `15`, since `Box` is passed by reference.

Chapter 7

Null-Reference Checking

The interpreter enforces variables being given values before they are referenced in expressions (i.e., not in the left-hand side of an assignment). A variable of primitive type must have been initialized before it was referenced. A record variable's attribute must have been given a value before it was accessed. Other attributes need not have been given a value yet.

Arrays are treated differently because null-reference checking of array elements is not possible pre-execution. Consequently, an array being indexed at a location where no value has been assigned will be only be caught during execution.

```
record Buf { data: int arr[8], size: int }

main: {
    var x: int;
    var b: Buf;
    var myArr: int arr[10];
    print(x);
    print(b.size);
    print(arr[0]);
}
```

The first two print statements result in errors raised during pre-execution since neither `x` nor `b.size` were given values before being passed, but the error in the last line, which results from the array's elements being uninitialized, will only be raised during execution.

Chapter 8

Quick Reference

Built-in Types

```
int   float   bool   char   str
basetype arr[d1, d2]
```

Declarations

```
var name: type;
var name = expr;
let name = expr;

record T {
    f1: t1,
    f2: t2
}

ReturnType f(p1: t1, p2: t2) {
    stmt1;
    stmt2;
    return expr;
}
void f(p1: t1){ }
```

Expressions

```
1
1.0
true
'c'
"hello world"
x
myArray[idx]
myRecord.attr

1 + 2
1 - 2
1 * 2
1 / 2
```

```
1 < 2
1 <= 2
1 > 2
1 >= 2
1 == 2
1 != 2
!true
true && false
true || false
(expr)
```

Statements

```
lval = expr;
print(expr);
scan(lval);
return;
return expr;

if (cond) { ... } else { ... }
while (cond) { ... }
repeat { ... } until (cond);
for (i; start; end; step) { ... }

myFunc();
```

Type Cast

```
cast(expr, "targetType")
```

8.1 Complete Example

```
record Vec2 { x: float, y: float }

void scale(v: Vec2, factor: float) {
    v.x = v.x * factor;
    v.y = v.y * factor;
}

float dot(a: Vec2, b: Vec2) {
    return a.x * b.x + a.y * b.y;
}

int fact(n: int) {
    var result = 1;
    for (i; 1; n; 1) {
        result = result * i;
    }
    return result;
}
```

```

main: {
    var u: Vec2;
    u.x = 3.0;
    u.y = 4.0;

    var v: Vec2;
    v.x = 1.0;
    v.y = 0.0;

    scale(u, 2.0);
    print(dot(u, v));

    let N = 5;
    print(fact(N));
    print(cast(fact(N), "str"));

    var i = 0;
    while (i < 3) {
        print(i);
        i = i + 1;
    }

    var x = 10;
    repeat {
        x = x - 3;
    } until (x < 0);
    print(x);
}
```